



The Harness

THE SHORT VERSION

Claude Code forgets everything between sessions. This is what it takes to make it stop — and what that buys once it has.

CONTENTS

- 00 Your agent forgets everything, and that is the whole problem
 - 01 The platform gives you eight things, and the rest is yours
 - 02 A session that cannot end badly
 - 03 Claims are the unit of work
 - 04 A document that fails its own build
 - 05 What this cannot do
 - . Getting it running
-



SECTION 00

Your agent forgets everything, and that is the whole problem

OPEN A NEW CLAUDE Code session and it knows nothing. Not "less than last time" — nothing. It has never seen this codebase, never made a decision here, never told you a number. Whatever it concluded yesterday left with the session.

Most of the time that is fine, because most of the time you are asking it to do something small and the answer arrives before the amnesia matters.

It stops being fine the moment the durable product of a session is a *conclusion* rather than a diff.

An analyst's session ends with a number. You asked a question, it ran a query, it reached an answer, you pasted the answer into a document somebody else will read. Nothing was committed. No task was left open. The session ends **clean**.

And the only thing that hour produced walks out with no record of where it came from. Six months later the figure is quoted back at you in a meeting and there is nothing — not the query, not the caveat you gave it, not the date the data was current.

This is what the kit is for. It is not a prompt library and it is not a collection of tips. It is a small amount of machinery that makes a session unable to end that way.

What is in it

Five components, and two of them are the point.

memory	notes as markdown in a git repository, written only through an MCP server, every write
session	a name per session so a later one can find it, an index, and a Stop guard that will not let a tur
journal	decisions, friction and ideas in one command — and a day that r
press	markdown into a finished magazine-format document, with a gate that fails the build when a stated fact stop
agreement	the operating rules, in one file the model reads o

Plus a menu of twelve further capabilities, none installed by default. That restraint is deliberate and section 05 explains it.



What it is not

It is not a framework. Nothing here wraps the model or intercepts your prompts. Every piece is an ordinary program that the platform already knows how to run — a hook, an MCP server, a command-line tool.

It is not clever. The most useful mechanism in the whole kit is a hash of the session id folded into two word lists. Section 02 is about why that is the good news. ■



SECTION 01

The platform gives you eight things, and the rest is yours

WORTH SEPARATING, because most people conflate them for months.

The platform provides: a context window with an agentic loop · a tool surface · a `CLAUDE.md` file loaded into every session · **hooks** that run at session start, before a tool call, after a tool call, and when the model tries to stop · skills · subagents · MCP · and permission modes.

The platform does not provide: memory that survives a session · any notion of a unit of work larger than a turn · any record of what a session concluded · any check that a claim in a document is true · any measure of what your own checks are missing.

Everything in this kit lives in the second list.

The distinction that matters

A rule in `CLAUDE.md` is **advice to a judgment engine**. It competes with every other rule in the file, with your phrasing, and with whatever the model believes about the task. It degrades gracefully and silently — when it stops being followed, nothing tells you.

A rule in a hook is a **program**. It runs whether or not the model agrees, and when it fails it fails loudly.

The ladder runs machine-blocking, then machine-visible, then human-remembered — and anything left on the bottom rung eventually stops happening.

That is the single most useful sentence to carry out of this document, and it generalises far beyond agents.

Where the interception points are

Four moments, and they are not equal.

- **Session start** — can inject context. Cannot stop anything.
- **Before a tool call** — **can deny**. This is the only point in the entire system where something can be *prevented*.
- **After a tool call** — can only advise. The thing already happened.
- **On stop** — can refuse to let the turn end.

A harness that wants to stop a bad thing must catch it before it happens. A harness that only notices afterwards has written a comment. ■



SECTION 02

A session that cannot end badly

THIS IS THE PIECE worth your attention even if you install nothing else.

An agent that finishes some work and stops produces two outcomes that look identical from outside. Either it is done, or it trailed off — "next I'll wire up the tests" — and there is no next, because the turn is over and nobody is coming back.

Both end with a confident paragraph.

It cannot read what the model is about to say

The constraint that shapes the whole design: at stop time the assistant's final message has not been written to the transcript yet. The guard cannot evaluate intent, or promises, or a claim of completion.

So it reads **state on disk** instead, and asks three questions. Is there declared work still open? Is there uncommitted work in this session's worktree? And — the one that matters — **did this session produce something, and never say where it stands?**

Six ways a turn is allowed to end

```
claim "<conclusion>" --source "<what it traces to>" [--does-not-establish "<...>"]
limit "<what you could not establish>"
discard "<why it is not worth keeping>"
add "<the next piece of work>"
hold "<what you need someone else to decide>"
clear
```

Two of these are unusual enough to call out.

–does-not-establish records the tempting conclusion a claim does *not* support. One field. It is what stops a correct number being used to argue something it cannot carry, and it is the field people notice first when they read a claim back a quarter later.

limit is a bounded *I do not know*, and it is a complete, legitimate ending. The temptation when building something like this is to make success the only acceptable answer. Do not. If the honest report is punished, you have not built a reporting channel — you have built an incentive to fabricate.

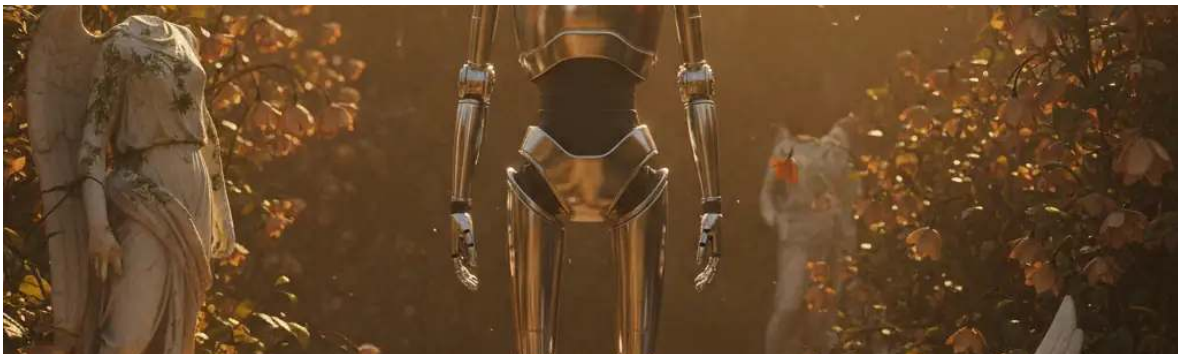
The three things a conclusion is not

The verbs above capture what a session *established*. They do not capture what it *decided*, what it *cost you*, or what it *turned up and set aside* — and none of those is a terminal state, so nothing forces them.

That gap is real and it is exactly the shape this document warns about everywhere else: left to the operating agreement, capture becomes a habit, and habits decay. So it gets a command:

```
journal note "<what>" --kind decision --why "<the reasoning>"
journal note "<what>" --kind friction
journal note "<what>" --kind idea
```

A decision without its reasoning is worth very little in three months, and the moment you make it is the cheapest it will ever be to write down.



The day rolls itself up

One command is still one command you have to run. The **roll-up** is not: it is regenerated on every stop, by the same hook that enforces the terminal state.

That placement is the whole design. Capture that depends on remembering to capture is the rung of the ladder that decays, so the roll-up is a **side effect of a gate that already runs on every single stop**. It is idempotent — twice equals once, which is what makes it safe to put in a hook at all — and it is backgrounded with its exit code discarded, because a broken journal must never be able to decide whether your turn is allowed to end.

What you get is a day you can actually read: decisions with their reasons, friction that will recur, ideas you set aside, and every conclusion the session recorded with its source and its `does_not_establish` intact.

The trigger is deliberately narrow

The obvious implementation fires whenever a session used any tool. That version was measured in the harness this was extracted from: it fired on **66 of 92 stops**, and about **three quarters of those bought nothing** — the agent had genuinely finished and was made to spend several turns saying so.

A guard that cries wolf gets routed around, and a guard that gets routed around is worse than no guard, because it still costs turns.

So the trigger is: a file was written, **or** a query was actually run — `psql`, `duckdb`, `bq`, `dbt`, a notebook execution, a dataset opened. Reading code is not analysis. Grepping is not analysis. `git status` is not analysis. Ambiguity resolves to *nothing happened*. ■



SECTION 03

Claims are the unit of work

A BUILDER'S UNIT of work is a task that completes. Their residue risk is a dropped task, and every project tool ever made is built for them.

An analyst's unit of work is a **claim that has to survive scrutiny**, and their residue risk is an unrecorded conclusion. The loop is not make → test → ship. It is:

claim → bind to established fact → defend → update when the foundation moves.

Once you see that, a lot of tooling looks wrong. Velocity is meaningless when the output is a defended position. A two-week timebox forces premature closure of an argument before its contradictions resolve. A severity-ranked backlog assumes you control the queue, when in fact work arrives as questions pinned to other people's calendars.

The kit therefore ships **no sprint machinery and no priority taxonomy**, and section 05 is honest about what else it does not have.



How memory closes its own loops

Notes are markdown files with YAML frontmatter in a git repository. There is no database. That one choice means the entire memory of the system is greppable, diffable, revertable and backup-able by tools you already have, and a corrupted search index is never data loss — because the index is derived and can always be thrown away.

Open threads are never hand-marked done. When a later note absorbs the work, **that note links to the thread**, and the thread drops off the open list the moment the link exists.

The direction is the whole trick. The closing note points at the target, not the reverse — so the note that absorbed the work becomes the thread's permanent home, and *where did that go?* answers itself by construction. ■



A document that fails its own build

THE PRESS TURNS markdown into a finished document. That part is ordinary. This is not:

```
<pre class="src"><code>def _handle_lines(root, arg, esc): p = _resolve_path(root, arg)
if not p.is_file(): raise ClaimError(f&quot;no such file: {arg}&quot;) n =
len(p.read_text(errors=&quot;replace&quot;).split(&quot;\n&quot;)) # A trailing
newline yields a final empty element; a file&#x27;s &quot;line count&quot; in # every
tool a reader would check against (wc -l) does not count it. if
p.read_text(errors=&quot;replace&quot;).endswith(&quot;\n&quot;): n -= 1 return
_fmt_int(n)</code></pre>
```

Every count, path and quoted function in a document you write with this is a **token in the markdown**, resolved against the working tree when the document is built. Delete a file the document references and the build exits non-zero and marks the broken claim on the page.

This page is doing it right now. There are 7 programs in the press, and 215 lines of one of them are the gate.

Why it exists

It is ported from a press that shipped a wrong number to paying readers for weeks.

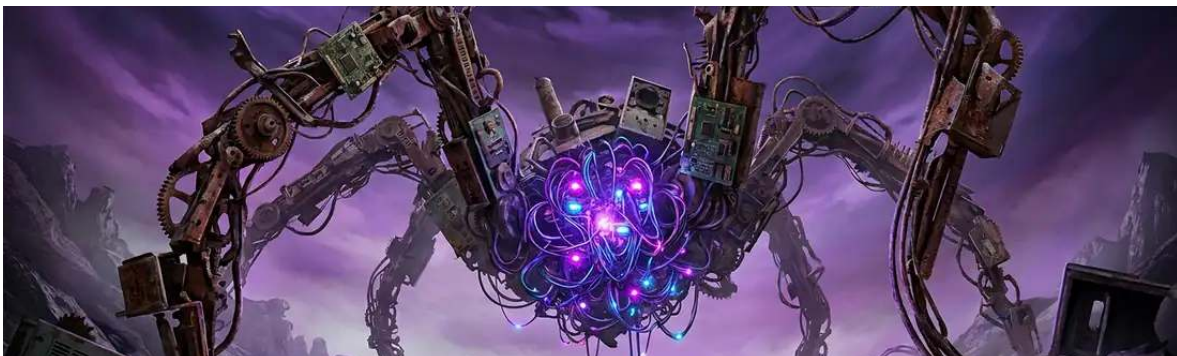
Two reader-facing claims lived in the *renderer* rather than in the gated source file. They survived three full review rounds and nine separate reviewer passes, because every one of those rounds was pointed at the source. Nobody was careless. The claims were simply outside the thing being checked.

A claim outside the gate is a claim outside the gate.

What it proves, and what it does not

It proves a file exists, a count is current, a quoted function still says what is quoted.

It proves **nothing about interpretation**. A document can pass this gate completely and still argue something the data does not support. It is a floor, not a ceiling, and treating a green build as a fact-check is exactly how it will mislead you. ■





SECTION 05

What this cannot do

THE SECTION THAT should decide whether you trust any of the above.

There is no independent reviewer

Everything reviewing your work here is Claude, including whatever reviews the work of the agent that produced it. A model asked to check its own output gives you a **correlated** second opinion: the framing it found natural when drafting, it finds natural when reviewing.

The kit's answer is held-out context — reviewers that see the artifact and the sources and **never the reasoning that produced them**, each given a distinct lens rather than running several identical ones. That catches unsourced claims, arithmetic that does not reproduce, and citations that do not say what is claimed.

It does not catch a framing error both of them find natural. Nothing built from one lineage does, and a tool that implies otherwise is selling assurance it does not have.

The menu is deliberately not installed

Twelve further capabilities are specified in menu/ — a claim spine with reverse dependency edges, a downstream blast-radius graph, an objection docket, a definition registry for the terms whose meaning you own. Every one is real and none is installed.

The reason is arithmetic. The menu holds well over 150 hours of proposals, and the binding constraint on any of this is not build hours, it is attention. **A thirty-hour component that never gets finished is worth zero.** Install the core, use it for a fortnight, then pick one.

The honest summary

This is a small amount of machinery that makes an agent remember what it concluded, refuse to end a session without saying where it stands, and refuse to publish a number that has stopped being true.

It is not intelligence. It is bookkeeping placed where forgetting is expensive. ■



APPENDIX

Getting it running

INSTALL .md is written to be pasted into a fresh Claude Code session, which will interview you for the parts no template can know and then install itself. It ends by running `<code>verify.py</code>`, which proves each component **on the machine it landed on** rather than asserting it from a config file.

That distinction earned itself immediately: `verify.py` caught two defects in the installer on its own first run.

Everything lands in `~/ .claude/` and a personal `~/harness/`. **Nothing touches a shared repository** — the one exception being `git config core.hooksPath`, which is per-clone and not shared, so a personal pre-commit guard costs your teammates nothing.

There are 18 Python modules in the core and a test suite that runs in about a second. Start there. ■